

```

    }
}

package nhnis.fw.common.util;

import java.lang.reflect.Field;
import java.lang.reflect.Method;
import java.math.BigDecimal;
import java.util.Map;

import lombok.extern.slf4j.Slf4j;

@Slf4j
public class MappingUtil {
    /**
     * Map 을 DTO 객체로 매핑하는 유틸리티 클래스
     *
     * <pre>
     * 사용 예시:
     * Map<String, Object> map = ...;
     * MyDto dto = MapToDtoUtil.mapToDto(map, MyDto.class);
     * </pre>
     */
    /**
     * Map 을 지정된 DTO 클래스의 인스턴스로 매핑합니다.
     * Map 의 키가 언더스코어 (TRT_BRC) 인 경우, DTO 필드명 (trtBrc) 으로 자동 변환됩니다.
     *
     * @param <T> DTO 타입
     * @param map 매핑할 Map 데이터
     * @param clazz 매핑할 DTO 클래스
     * @return 매핑된 DTO 인스턴스
     * @throws Exception 매핑 중 오류 발생 시 예외
     */
    public static <T> T mapToDto(Map<String, Object> map, Class<T> clazz) throws
    Exception {
        if (map == null || clazz == null) {
            return null;
        }

        T dto = clazz.getDeclaredConstructor().newInstance();

        // DTO 의 모든 필드 순회
        for (Field field : clazz.getDeclaredFields()) {
            String fieldName = field.getName();

            // Map 에서 매칭될 키 찾기 (카멜케이스 또는 언더스코어 변환)
            String mapKey = findMapKey(map, fieldName);

            if (mapKey == null) {
                continue;
            }

            Object value = map.get(mapKey);

            // 값이 null 이면 스킵
            if (value == null) {
                continue;
            }

            // Setter 메서드 이름 생성 (set + 필드명 첫글자 대문자)
            String setterName = "set" +
            Character.toUpperCase(fieldName.charAt(0)) + fieldName.substring(1);

```

```

        // Setter 메서드 찾기
        Method setter = findSetterMethod(clazz, setterName, field.getType());

        if (setter != null) {
            Object convertedValue = convertValue(value, field.getType());
            setter.invoke(dto, convertedValue);
        }

        return dto;
    }

    /**
     * Map 에서 DTO 필드명과 매칭되는 키를 찾습니다.
     * 1. 정확한 필드명으로 검색
     * 2. 언더스코어 형식 (TRT_BRC) 으로 변환하여 검색
     */
    private static String findMapKey(Map<String, Object> map, String fieldName) {
        // 1. 정확한 필드명으로 검색 (trtBrc)
        if (map.containsKey(fieldName)) {
            return fieldName;
        }

        // 2. 카멜케이스 → 언더스코어 변환 후 검색 (trtBrc → TRT_BRC)
        String snakeCaseKey = camelToSnake(fieldName);
        if (map.containsKey(snakeCaseKey)) {
            return snakeCaseKey;
        }

        // 3. 대문자 카멜케이스로도 시도 (trtBrc → TRTBRC)
        String upperCamelKey = fieldName.toUpperCase();
        if (map.containsKey(upperCamelKey)) {
            return upperCamelKey;
        }

        return null;
    }

    /**
     * 카멜케이스를 언더스코어 대문자로 변환합니다.
     * 예: trtBrc → TRT_BRC, trtmnEno → TRTMN_ENO
     */
    private static String camelToSnake(String camelCase) {
        if (camelCase == null || camelCase.isEmpty()) {
            return camelCase;
        }

        StringBuilder result = new StringBuilder();
        for (int i = 0; i < camelCase.length(); i++) {
            char ch = camelCase.charAt(i);
            if (Character.isUpperCase(ch)) {
                if (i > 0) {
                    result.append("_");
                }
                result.append(ch);
            } else {
                result.append(Character.toUpperCase(ch));
            }
        }
        return result.toString();
    }
}

```

```

/**
 * 지정된 이름과 타입을 가진 Setter 메서드를 찾습니다.
 */
private static Method findSetterMethod(Class<?> dtoClass, String methodName,
Class<?> paramType) {
    try {
        // 정확한 파라미터 타입으로 메서드 찾기
        return dtoClass.getMethod(methodName, paramType);
    } catch (NoSuchMethodException e) {
        // 타입이 다른 호환 가능한 타입 찾기 (예: Integer -> int)
        for (Method method : dtoClass.getMethods()) {
            if (method.getName().equals(methodName) &&
method.getParameterCount() == 1) {
                Class<?>[] paramTypes = method.getParameterTypes();
                if (isAssignable(paramTypes[0], paramType)) {
                    return method;
                }
            }
        }
    }
    return null;
}

/**
 * 두 타입이 호환 가능한지 확인합니다.
 */
private static boolean isAssignable(Class<?> targetType, Class<?> sourceType) {
    if (targetType == sourceType) {
        return true;
    }

    // 기본 타입과 래퍼 타입 호환
    if (targetType.isPrimitive() || sourceType.isPrimitive()) {
        return isPrimitiveCompatible(targetType, sourceType);
    }

    // Number 의 하위 타입 호환 (Integer -> Number 등)
    if (Number.class.isAssignableFrom(targetType) &&
Number.class.isAssignableFrom(sourceType)) {
        return true;
    }

    // String 으로 변환 가능한 경우
    if (targetType == String.class) {
        return true;
    }

    return targetType.isAssignableFrom(sourceType);
}

/**
 * 기본 타입 호환성 확인
 */
private static boolean isPrimitiveCompatible(Class<?> targetType, Class<?>
sourceType) {
    // int <-> Integer
    if (targetType == int.class && sourceType == Integer.class) return true;
    if (targetType == Integer.class && sourceType == int.class) return true;

    // long <-> Long
    if (targetType == long.class && sourceType == Long.class) return true;
    if (targetType == Long.class && sourceType == long.class) return true;
}

```

```

// double <-> Double
if (targetType == double.class && sourceType == Double.class) return true;
if (targetType == Double.class && sourceType == double.class) return true;

// float <-> Float
if (targetType == float.class && sourceType == Float.class) return true;
if (targetType == Float.class && sourceType == float.class) return true;

// boolean <-> Boolean
if (targetType == boolean.class && sourceType == Boolean.class) return true;
if (targetType == Boolean.class && sourceType == boolean.class) return true;

// char <-> Character
if (targetType == char.class && sourceType == Character.class) return true;
if (targetType == Character.class && sourceType == char.class) return true;

return false;
}

/**
 * 값을 대상 타입으로 변환합니다.
 */
private static Object convertValue(Object value, Class<?> targetType) {
    if (value == null) {
        return null;
    }

    // 이미 올바른 타입이면 그대로 반환
    if (targetType.isInstance(value)) {
        return value;
    }

    try {
        // String 변환
        if (targetType == String.class) {
            return value.toString();
        }

        // Integer 변환
        if (targetType == int.class || targetType == Integer.class) {
            if (value instanceof Number) {
                return ((Number) value).intValue();
            }
            return Integer.parseInt(value.toString());
        }

        // Long 변환
        if (targetType == long.class || targetType == Long.class) {
            if (value instanceof Number) {
                return ((Number) value).longValue();
            }
            return Long.parseLong(value.toString());
        }

        // Double 변환
        if (targetType == double.class || targetType == Double.class) {
            if (value instanceof Number) {
                return ((Number) value).doubleValue();
            }
            return Double.parseDouble(value.toString());
        }
    }
}

```

```

// Float 변환
if (targetType == float.class || targetType == Float.class) {
    if (value instanceof Number) {
        return ((Number) value).floatValue();
    }
    return Float.parseFloat(value.toString());
}

// Boolean 변환
if (targetType == boolean.class || targetType == Boolean.class) {
    if (value instanceof Boolean) {
        return (Boolean) value;
    }
    return Boolean.parseBoolean(value.toString());
}

// BigDecimal 변환
if (targetType == BigDecimal.class) {
    if (value instanceof Number) {
        return new BigDecimal(value.toString());
    }
    return new BigDecimal(value.toString());
}

// 그 외 타입은 toString() 으로 변환 시도
return value.toString();

} catch (Exception e) {
    log.warn("값 변환 실패: target={}, value={}, error={}",
targetType.getSimpleName(), value, e.getMessage());
    return null;
}
}

package nhnis.fw.common.util;

import java.lang.reflect.Array;

public class NumberUtil {
    public static long max(long... array) {
        validateArray(array);
        long max = array[0];
        for (int j = 1; j < array.length; j++) {
            if (array[j] > max)
                max = array[j];
        }
        return max;
    }

    public static int max(int... array) {
        validateArray(array);
        int max = array[0];
        for (int j = 1; j < array.length; j++) {
            if (array[j] > max)
                max = array[j];
        }
        return max;
    }

    public static short max(short... array) {
        validateArray(array);

```

```

        short max = array[0];
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max)
                max = array[i];
        }
        return max;
    }

    public static byte max(byte... array) {
        validateArray(array);
        byte max = array[0];
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max)
                max = array[i];
        }
        return max;
    }

    public static double max(double... array) {
        validateArray(array);
        double max = array[0];
        for (int j = 1; j < array.length; j++) {
            if (Double.isNaN(array[j]))
                return Double.NaN;
            if (array[j] > max)
                max = array[j];
        }
        return max;
    }

    public static float max(float... array) {
        validateArray(array);
        float max = array[0];
        for (int j = 1; j < array.length; j++) {
            if (Float.isNaN(array[j]))
                return Float.NaN;
            if (array[j] > max)
                max = array[j];
        }
        return max;
    }

    private static void validateArray(Object array) {
        ValidateUtil.isTrue((array != null), "The Array must not be null", new
Object[0]);
        ValidateUtil.isTrue((Array.getLength(array) != 0), "Array cannot be empty.",
new Object[0]);
    }

    public static long min(long a, long b, long c) {
        if (b < a)
            a = b;
        if (c < a)
            a = c;
        return a;
    }

    public static int min(int a, int b, int c) {
        if (b < a)
            a = b;
        if (c < a)
            a = c;
    }

```

```

        return a;
    }

    public static short min(short a, short b, short c) {
        if (b < a)
            a = b;
        if (c < a)
            a = c;
        return a;
    }

    public static byte min(byte a, byte b, byte c) {
        if (b < a)
            a = b;
        if (c < a)
            a = c;
        return a;
    }

    public static double min(double a, double b, double c) {
        return Math.min(Math.min(a, b), c);
    }

    public static float min(float a, float b, float c) {
        return Math.min(Math.min(a, b), c);
    }

    public static long max(long a, long b, long c) {
        if (b > a)
            a = b;
        if (c > a)
            a = c;
        return a;
    }

    public static int max(int a, int b, int c) {
        if (b > a)
            a = b;
        if (c > a)
            a = c;
        return a;
    }

    public static short max(short a, short b, short c) {
        if (b > a)
            a = b;
        if (c > a)
            a = c;
        return a;
    }

    public static byte max(byte a, byte b, byte c) {
        if (b > a)
            a = b;
        if (c > a)
            a = c;
        return a;
    }

    public static double max(double a, double b, double c) {
        return Math.max(Math.max(a, b), c);
    }
}

```